API, Sandbox и залог

Для расчётов площадки рассматриваем ЮKassa. API кошелька ЮMoney — другой продукт.

https://api.yookassa.ru/v3/ · JSON · Basic Auth: ID магазина / шлюза + ключ

1
Создать оплату
POST /payments
capture: true — списание

2
Подтвердить
Арендатор переходит
по confirmation_url

3
Проверить результат
GET /payments/{id}
или проверенный webhook

/deals
связать расчёты

/refunds
вернуть оплату

/payouts
выплатить владельцу

/receipts
передать данные чека

Оплата подтверждена по succeeded; возврат на сайт сам по себе её не подтверждает.

При повторе POST сохраняем Idempotence-Key и тело: окно 24 ч. При таймауте сначала сверяем статус.

Sandbox: проверяем путь без реальных денег

Для «Безопасной сделки» нужна тестовая интернет-площадка ЮKassa.

СЕЙЧАС В КОДЕ

Локальный Simulator

Работает без ЮKassa.

Возвращает sim_… и заданный успешный результат.

Проверяет нашу логику.

СЛЕДУЮЩИЙ ЭТАП

Sandbox ЮKassa

Настоящие API-запросы.

Тестовые shopId, ключ и карты; в ответе test: true.

Деньги не переводятся.

ПОСЛЕ ПРИЁМКИ

Реальные платежи

Подключённый продукт.

Боевые реквизиты; в ответе test: false.

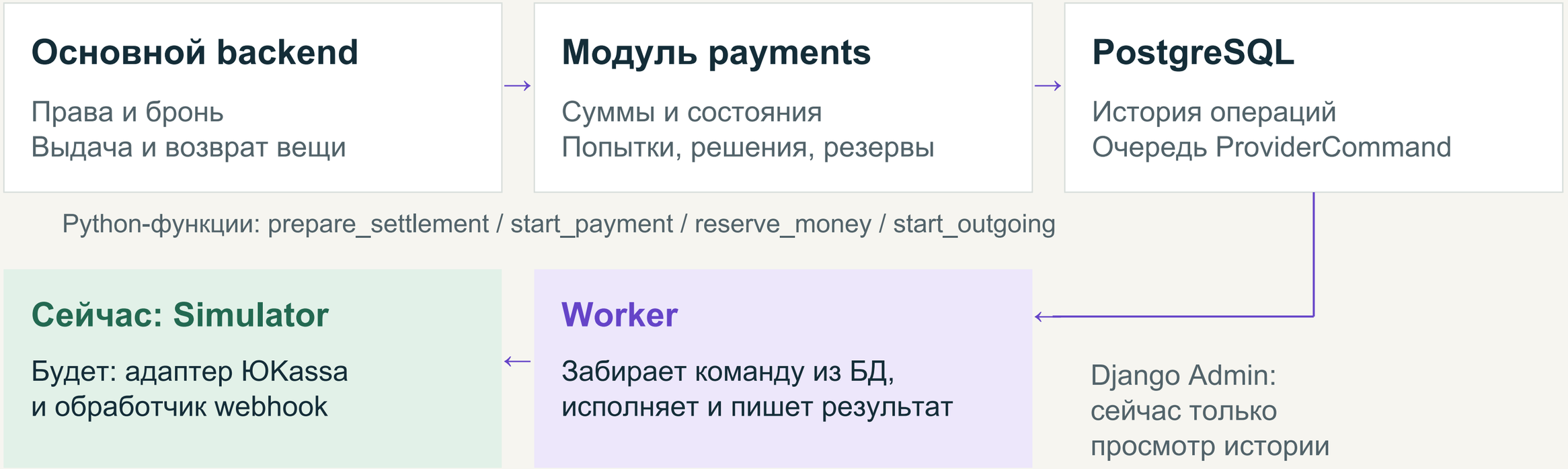
Движутся реальные деньги.

Что прогнать в Sandbox

Успех и отказ · Повтор и таймаут · Оплачена только одна часть · Три исхода залога

Архитектура: Django-модуль и очередь в БД

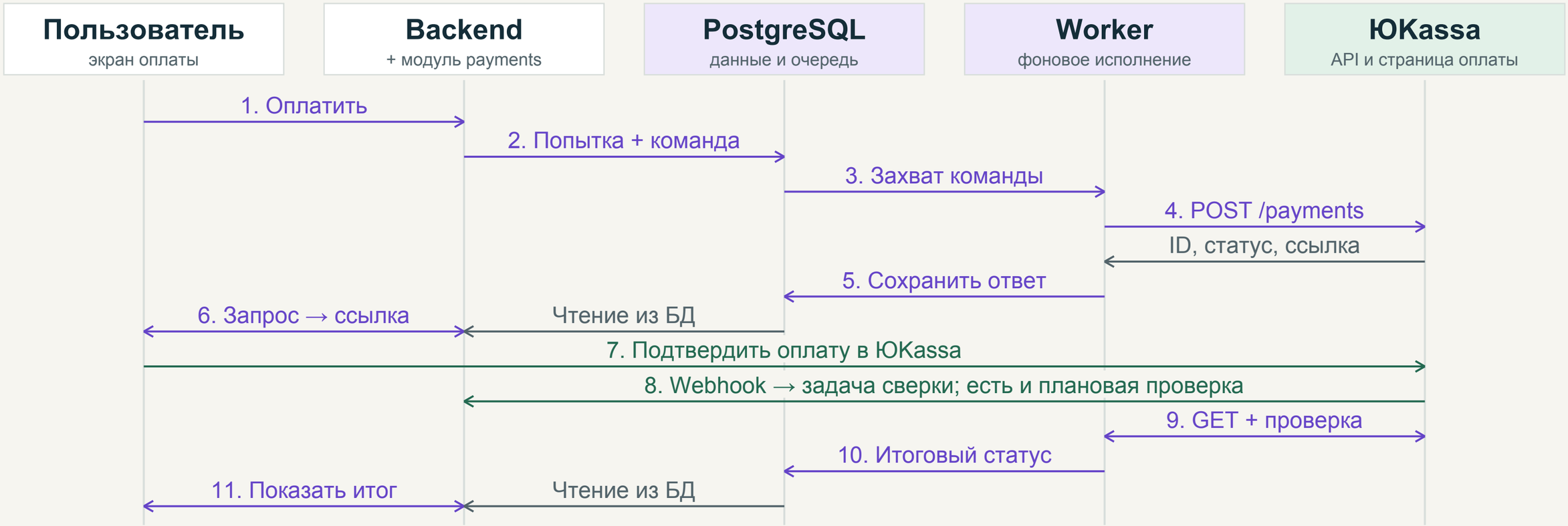
Контракт подключения: payments внутри основного backend, данные — в общей PostgreSQL.



Команда сохраняется до исполнения. Вызов провайдера — вне транзакции БД; повтор защищён ключом.

Как выполняется оплата: от кнопки до статуса

Целевая схема после подключения ЮKassa. Аренда и залог проходят этот путь отдельно.



Залог: решение → резерв → команды для ненулевых частей → тот же worker → возврат / выплата.

Sandbox → live: тот же сценарий оплаты

После реализации интеграции, в рамках одного согласованного продукта ЮKassa.

СОХРАНЯЕМ

Код работы с API и адрес /v3/

Расчёты, очередь и защиту от повторов

Проверку подтверждённых статусов

НАСТРАИВАЕМ ДЛЯ LIVE

Боевые реквизиты и секреты

HTTPS-уведомления, чеки и методы оплаты

Отдельный учёт test/live и новые ID

Сначала: адаптер ЮKassa, webhook, чеки и интеграция выдачи / отмены с основным backend.

Пуск: согласование → сценарии Sandbox → боевые настройки → контрольная оплата, возврат и выплата.

Залог: отдельная оплата и три исхода

Залог реально списывается отдельно от аренды. Наценки площадки на него нет.

Аренда: 960 ₽

800 ₽ владельцу + 160 ₽ наценка площадки

Залог: 3 000 ₽

Возвратная сумма, учёт отдельно от аренды

Распределение залога	Арендатору	Владельцу
Претензий нет	**3 000 ₽**	**0 ₽**
Подтверждённый ущерб 500 ₽	**2 500 ₽**	**500 ₽**
Компенсация на весь залог	**0 ₽**	**3 000 ₽**

Сумму подтверждают обе стороны или поддержка. Расходы провайдера покрывает площадка; долг за время учитывается отдельно от залога.

Что уже реализовано для залога

Денежные обязательства и попытки исполнения уже есть; банковское исполнение пока имитируется.

01

Создать начисления

prepare_settlement

Сохраняет условия. Аренда и залог — отдельные Charge.

02

Принять оплату

start_payment

Создаёт попытку и команду. Worker вызывает Simulator.

03

Распределить залог

reserve_money

После успешной оплаты: решение + резервы на возврат и компенсацию.

04

Исполнить части

start_outgoing

Для каждого резерва — отдельная попытка возврата или выплаты.

Защита денег

Суммы — в целых копейках.
Нельзя повторно распределить один залог.

Граница реализации

Резерв — запись в БД, а не банковский холд.
Пересмотр резервов пока не реализован.

Начисления: services.py Резервы: reservations.py Исполнение: outgoing.py

Для реального залога нужно согласовать схему

Наличие API возврата и выплаты ещё не подтверждает весь сценарий P2P-залога.

«БЕЗОПАСНАЯ СДЕЛКА»

В одной сделке — один активный или успешный платёж.

Разница «оплата − сумма владельцу» должна быть больше комиссии эквайринга. Схему «3 000 ₽ → все 3 000 ₽ владельцу» стандартная инструкция не подтверждает.

КАНДИДАТ ДЛЯ СОГЛАСОВАНИЯ

Обычный приём залога → возврат + отдельная выплата владельцу.

Площадка обеспечивает деньги на возвраты, выплаты и комиссии. Балансы раздельные. Назначение залога и условия подключения нужно подтвердить у ЮKassa.

Холд — другой вариант: capture: false, срок до 7 дней; модель проекта требует 14 + 7 суток.

В Safe Deal возврат возможен до выплаты / закрытия; учитываем фактический expires_at.

Если выбрана Safe Deal: сначала возврат остатка, затем компенсация. Эта зависимость в модуле ещё не реализована.

Итог: механизм понятен, подключение впереди

Исследование отвечает на вопросы API, Sandbox, перехода в live и внутреннего устройства залога.

API	Оплаты, возвраты, выплаты, статусы и повторы разобраны.
Sandbox → live	Переход спроектирован; настоящий Sandbox ещё не пройден.
Архитектура	Django + PostgreSQL + очередь + worker; сейчас Simulator.
Залог	Расчёт и резервы есть; банковская схема требует согласования.

Следующие шаги

1. Согласовать залог и комиссии 2. Подключить ЮKassa 3. Пройти все сценарии Sandbox